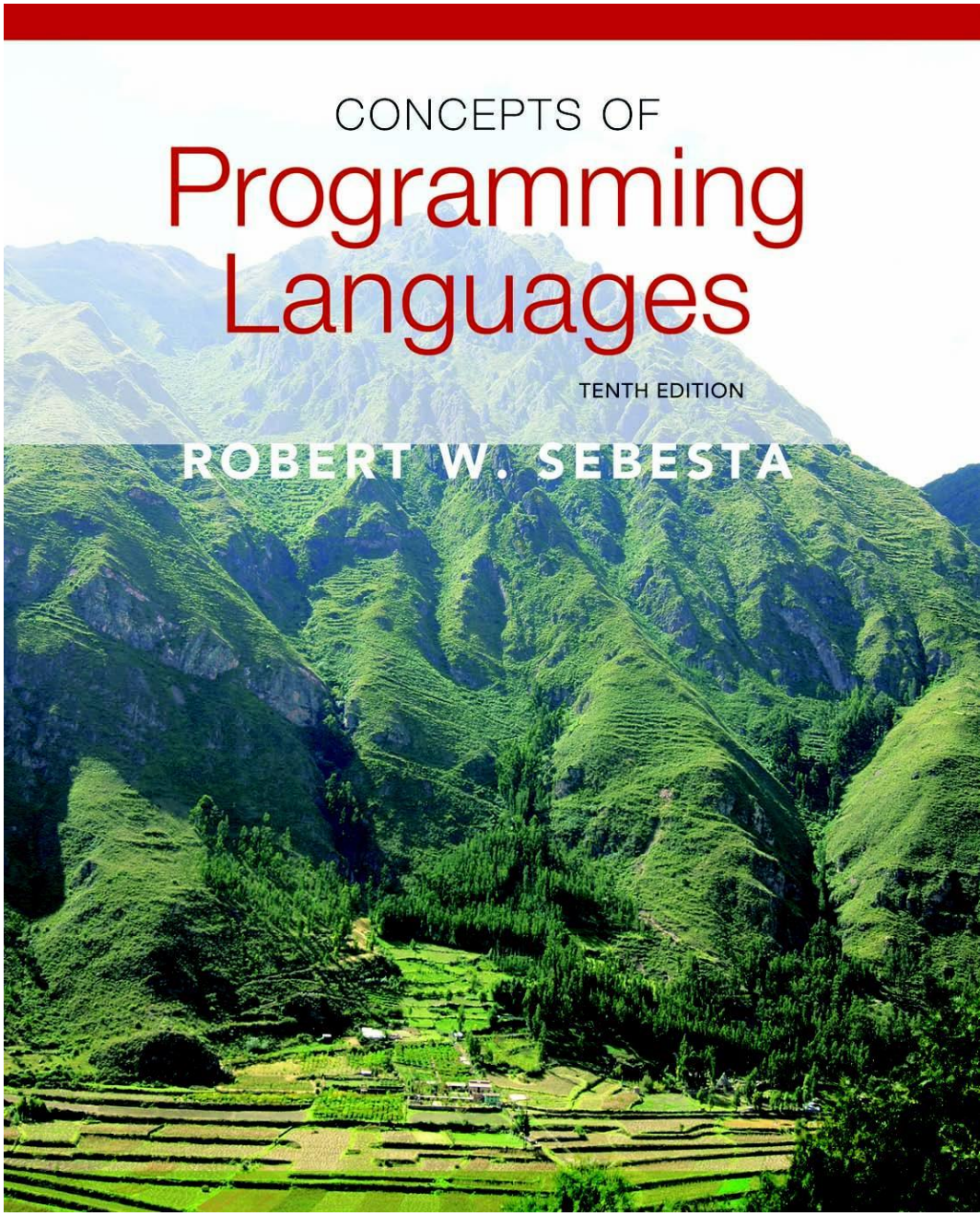
Chapter 5

Names, Bindings, and Scopes



Chapter 5 Topics

- Names
- Variables
- The Concept of Binding
- Scope
- Scope and Lifetime
- Referencing Environments
- Named Constants

Copyright © 2012 Addison-Wesley. All rights reserved.

Names, Bindings, and Scopes

- Most important feature of high-level languages:
  - programmers are able to give **names** to program entities
    - variable, function, constant, type, …
  - most names are called identifiers
  - symbols (like '+') can also be names

- A **binding** is an association between two things, such as a name and the thing it names

- The **scope** of a binding is the part of the program (textually) in which the binding is active

Copyright © 2012 Addison-Wesley. All rights reserved.

Names

- Design issues for names:
  - Are names case sensitive?
  - Are special words reserved words or keywords?

- Length
  - If too short, they cannot be meaningful
  - Language examples:
    - **FORTRAN 95**: maximum of 31
    - **C99**: no limit but only the first 63 are significant
    - **C#, C++, Java, Haskell**: no limit, all are significant

Copyright © 2012 Addison-Wesley. All rights reserved.

Names (continued)

- Special characters
  - **PHP:** all variable names must begin with dollar signs
  - **Perl:** all variable names begin with special characters, which specify the variable's type
    - $: scalar, @: array, or %: hash
  - **Ruby:** variable names that begin with:
    - `@` are instance variables;
    - `@@` are class variables

Copyright © 2012 Addison-Wesley. All rights reserved.

Names (continued)

- Case sensitivity
  - **Disadvantage**: readability (names that look alike are different)
    - Names in the **C-based languages** are case sensitive
    - Names in others are not
    - Worse in **C++, Java, and C#** because predefined names are mixed case
      - (e.g. indexOutOfBoundsException)

Copyright © 2012 Addison-Wesley. All rights reserved.

Names (continued)

- Special words
  - An aid to readability; used to delimit or separate statement clauses
  - A *keyword* is a word that is special only in certain contexts,
    - e.g., in Fortran:
      - `Real VarName` (`Real` *is a data type followed with a name, therefore* `Real` *is a keyword)*
      - `Real = 3.4` (*`Real` is a variable)*
  - A *reserved word* is a special word that cannot be used as a user-defined name

Copyright © 2012 Addison-Wesley. All rights reserved.

Variables

- A variable is an abstraction of a memory cell

- Variables can be characterized with their attributes:
  1. Name
  2. Address
  3. Type
  4. Value
  5. Lifetime
  6. Scope

Copyright © 2012 Addison-Wesley. All rights reserved.

Variables Attributes

2. Address – the memory address with which it is associated
   - A variable may have different addresses at different times during execution
     - local variables in subprograms – different addresses allocated at different calls
   - Multiple variables can have the same address
     - more than one variable accessing the same memory location → aliases
     - Aliases are created via pointers, reference variables, subprogram parameters, C and C++ unions
     - Aliases are harmful to readability (program readers must remember all of them)

Copyright © 2012 Addison-Wesley. All rights reserved.

Variables Attributes (continued)

3. Type determines the range of values of variables and the set of operations that are defined for values of that type

C types:

- `int, char, long`, …
- `float, double`
- structures:
  `struct, union`
- pointers
- arrays

Haskell types:

- `Bool, Int, Float`, …
- `Char, String`
- tuples, (N-tuples), records
- lists,
- functions

Python types:

- `bool, int, float, complex`
- `str, bytes, tuple, list, set, dict`
- classes, functions

Copyright © 2012 Addison-Wesley. All rights reserved.

Variables Attributes (continued)

4. Value – the contents of the memory cell(s) with which the variable is associated
    - Constants, parameters, function return values, operator results, …

- The l-value of a variable is its address
- The r-value of a variable is its value

Copyright © 2012 Addison-Wesley. All rights reserved.

Variables

- A variable is an abstraction of a memory cell

- Variables can be characterized with their attributes:
  1. Name
  2. Address
  3. Type
  4. Value
  5. Scope
  6. Lifetime

Copyright © 2012 Addison-Wesley. All rights reserved.

Referencing Environments

The referencing environment of a statement is the collection of all identifiers that are visible in the statement

```
struct Person { ··· } x;
int f(int a) {
   double y;
   int x;
     ···    /** 1 **/
}
int main() {
   double a;
     ···    /** 2 **/
}
```

E(1) = { struct Person → type,
x → int, f → func, a → int,
y → double }

E(2) = { struct Person → type,
x → struct Person, f → func,
a → double, main → func }

Copyright © 2012 Addison-Wesley. All rights reserved.

Blocks

- Program blocks define the scope of identifiers declared inside. For variables, they also define the lifetime.

- Languages may have different block structures:

  C: function definitions and command blocks ({ … }) define local scopes. Also, each source code define a block

  Java: class definitions, class member function definitions, block commands define local scopes. Nested function definitions and namespaces possible.

  Haskell: `'let definitions in expression'`, `'expression where definitions'` define block expression (the definitions have a local scope and not accessible outside of the expression)

Copyright © 2012 Addison-Wesley. All rights reserved.

Monolithic Block Structure

- Whole program is a block
- All identifiers have global scope starting from the definition
- **Cobol** is a monolithic block structure language

```
int x;
int y;
...
...
```

- In a long program with many identifiers, they share the same scope and need to be distinct

Copyright © 2012 Addison-Wesley. All rights reserved.

Flat Block Structure

- Program contains the global scope and only a single level local scope of function definitions. No further nesting is possible.
- **Fortran** has flat block structure

```
int x;
int y;
int f()
    { int a;
      double b;
      ...
    }
int g()
    { int a;
      double b;
      ...
    }
...
```

Nested Block Structure

- Multiple blocks with nested local scopes can be defined.
- **Pascal, Java, C (gcc)** have nested block structure

```
int x;
int f()
    { int a;
      double g()
          { int x;
            ...
          }
    ...
    }
int g()
    { int h()
          { int x;
            ...
          }
      ...
    }
...
```

Hiding

- Identifiers defined in the inner local block hides the outer block identifiers with the same name during their scope. They cannot be accessed within the inner block

```
int x,y;
int f(double x){
     ...          // parameter x hides global x in f()
}
int g(double a){
    int y;        // local y hides global y in g()
    double f;     // local f hides global f() in g()
     ...
}
int main(){
    int y;        // local y hides global y in main()
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Variable Attributes: Scope

5. Scope of a variable is the range of statements over which it is visible

- The local variables of a program unit are those that are declared in that unit
- The nonlocal variables of a program unit are those that are visible in the unit but not declared there
- Global variables are a special category of nonlocal variables

- The scope rules of a language determine how references to names are associated with variables

Copyright © 2012 Addison-Wesley. All rights reserved.

Static Scope

- Based on program text
- To connect a name reference to a variable, you (or the compiler) must find the declaration
- Search process: search declarations
  - first locally,
  - then in increasingly larger enclosing scopes,
  - until one is found for the given name
- Enclosing static scopes (to a specific scope) are called its static ancestors
  - the nearest static ancestor is called a static parent
- Some languages allow nested subprogram definitions, which create nested static scopes
  - e.g., Ada, JavaScript, Common LISP, Scheme, Fortran 2003+, F#, Python

Copyright © 2012 Addison-Wesley. All rights reserved.

Scope (continued)

- Variables can be hidden from a unit by having a "closer" variable with the same name
- What will happen if the same identifier name is used again?
  - C forbids reuse of same identifier name in the same scope. Same name can be used in different nested blocks. The identifier inside **hides** the outside identifier.

```
double f,y;
int f(){   X error
   ...
}
int y;     X error
```

```
double y;
int f(){
   double f;  ✓ OK
   int y;     ✓ OK
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Referencing Environments

- The referencing environment of a statement is the collection of all identifiers that are visible in the statement
  - In a static-scoped language, it is the local variables plus all of the visible variables in all of the enclosing scopes
  - In a dynamic-scoped language, it is the local variables plus all visible variables in all active subprograms
    - A subprogram is active if its execution has begun but has not yet terminated

Copyright © 2012 Addison-Wesley. All rights reserved.

Referencing Environments

```
struct Person { ··· } x;
int f(int a) {
   double y;
   int x;
    ...    /** 1 **/
}
int main() {
   double a;
    ...    /** 2 **/
}
```

Static-scope

E(1) = { struct Person → type,
x → int, f → func, a → int,
y → double }

E(2) = { struct Person → type,
x → struct Person, f → func,
a → double, main → func }

Copyright © 2012 Addison-Wesley. All rights reserved.

Dynamic Scope

- Based on calling sequences of program units, not their textual layout (temporal versus spatial)
- References to variables are connected to declarations by searching back through the chain of subprogram calls that forced execution to this point

Copyright © 2012 Addison-Wesley. All rights reserved.

Scope Example

```
function big() {
   function sub1(){
        var x = 7;
        sub2();
   }
   function sub2() {
        var y = x;
        var z = 3;
   }
   var x = 3;
   sub1();
}
```

big calls sub1
sub1 calls sub2
sub2 uses x

Copyright © 2012 Addison-Wesley. All rights reserved.

Scope Example

```
function big() {
   function sub1(){
        var x = 7;
        sub2();
   }
   function sub2() {
        var y = x;
        var z = 3;
   }
   var x = 3;
   sub1();
}
```

`big` calls `sub1`
`sub1` calls `sub2`
`sub2` uses `x`

- Static scoping
  - Reference to `x` in `sub2` is to `big`'s `x`

Scope Example

```
function big() {
   function sub1(){
        var x = 7;
        sub2();
   }
   function sub2() {
        var y = x;
        var z = 3;
   }
   var x = 3;
}
```

`big` calls `sub1`
`sub1` calls `sub2`
`sub2` uses `x`

- Static scoping
  - Reference to `x` in `sub2` is to `big`'s `x`
- Dynamic scoping
  - Reference to `x` in `sub2` is to `sub1`'s `x`

Static Scope

```
int x=1,y=2;
int f(int y){
   y=x+y;              // x global y local
   return x+y;
}
int g(int a){          // x local y global
   int x=3; y=x+x+a; x=x+y; y=f(x);
   return x;
}
int main(){            // x global y local
   int y=0; int a=10;
   x=a+y;   y=x+a;  a=f(a); a=g(a);
   return 0;
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Dynamic Scope

```
1.  int x=1,y=2;
2.  int f(int y){
3.    y=x+y;
4.    return x+y;
5.  }
6.  int g(int a){
7.    int x=3;
8.    y=x+x+a; x=x+y;
9.    y=f(x);
10.   return x;
11. }
12. int main(){
13.   int y=0; int a=10;
14.   x=a+y; y=x+a;
15.   a=f(a); a=g(a);
16.   return 0;
17. }
```

	Trace	Environment (without functions)
	initial	{x:GL, y:GL }
12	call main	{x:GL, y:main, a:main }
15	call f(10)	{x:GL, y:f , a:main }
4	return f : 30	back to environment before f
15	in main	{x:GL, y:main, a:main }
15	call g(30)	{x:g, y:main, a:g }
9	call f(39)	{x:g, y:f, a:g }
4	return f : 117	back to environment before f
9	in g	{x:g, y:main, a:g }
10	return g : 39	back to environment before g
15	in main	{x:GL, y:main, a:main}
16	return main	x:GL=10, y:GL=2, y:main=117, a:main=39

Copyright © 2012 Addison-Wesley. All rights reserved.

The Concept of Binding

- Variables are declared once, used many times

  Definition of a variable ⇔ used position of a variable

  Binding occurrence ⇔ applied occurrence

  `int n=7;` ⇔ `a=n+1;`

Binding is to find the corresponding binding occurrence (definition/declaration) for an applied occurrence (usage) of an identifier

Binding time is the time at which a binding takes place.

Copyright © 2012 Addison-Wesley. All rights reserved.

Static and Dynamic Binding

- A binding is *static* if it first occurs before run time and remains unchanged throughout program execution.
  - C, Haskell, Java, …
- A binding is *dynamic* if it first occurs at run time or can change during execution of the program
  - Python, Javascript, PHP, Ruby

we talk about the binding of identifiers to the variables they name

Copyright © 2012 Addison-Wesley. All rights reserved.

Type Binding

- How is a type specified?
- When does the binding take place?
  - If static, the type may be specified by either an explicit or an implicit declaration

Copyright © 2012 Addison-Wesley. All rights reserved.

Static Type Binding – Variable Declarations

- explicit declaration (by statement)
  - a program statement used for declaring the types of variables

```
int i; char c;
```

- implicit declaration (by first appearance)
  - associating variables with types through default conventions, rather than declaration statements
  - **Fortran**: if no data type is given for an identifier
    - if name starting with `I,J,K,L,M,N` → `Integer`
    - otherwise → `Real`

- Both creates *static binding* to types

Copyright © 2012 Addison-Wesley. All rights reserved.

Static Type Binding – Variable Declarations

- Implicit declarations may lead to problems. Some solutions:
  - **Perl**: requires names for specific types to begin with a particular special character

```
$apple: scalar
@apple: array
%apple: hash
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Static Type Binding – Variable Declarations

- Some implicit declarations use context. The context is the type of the value assigned to the variable in a declaration statement
  - Haskell, Go, ML
    - First appearance of a name is the basis for determining its type
  - C#: `var` declaration of a variable must include an initial value

```
var sum = 0;   (int)
var total = 0.0;   (float)
var name = "Fred";   (string)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Dynamic Type Binding

- Type of a variable:
  - is not specified by a declaration statement
  - cannot be determined by the spelling of its name
  - is specified through an assignment statement e.g., JavaScript

- Advantage: Allows programming flexibility
- Example languages: Python, Javascript, PHP, Ruby
  - e.g. In JavaScript
    - **`list = [10.2, 5.1, 0.0]`**
      - `list` is a single dimensioned array of length 3.
    - **`list = 73`**
      - `list` is now an integer.

Copyright © 2012 Addison-Wesley. All rights reserved.

Dynamic Type Binding

Disadvantages:

1. less reliable: compiler cannot check and enforce types
   - **Example**: Suppose `i` and `x` are integer variables, and `y` is a floating-point:
     - the correct statement is `i=x`
     - but by a typing error `i=y` is typed.
   - In a dynamic type binding language, this error cannot be detected by the compiler.
     - `i` is changed to float during execution.
     - The value of `i` becomes erroneous.

Copyright © 2012 Addison-Wesley. All rights reserved.

Dynamic Type Binding

Disadvantages:

2. Cost:
   - Type checking must be done at run-time.
   - Every variable must have a run-time descriptor to maintain current type.

Languages that use dynamic type bindings are usually implemented as interpreters

Copyright © 2012 Addison-Wesley. All rights reserved.

Variable Attributes: Lifetime

6. Lifetime of a variable is the time during which it is bound to a particular memory cell

- Storage Bindings & Lifetime
  - Allocation – process of taking the memory cell from a pool of available memory
  - Deallocation – process of placing the memory cell that has been unbound from a variable back into the pool of available memory

Copyright © 2012 Addison-Wesley. All rights reserved.

Categories of Variables by Lifetimes

1. Static
2. Stack-dynamic
3. Explicit heap-dynamic
4. Implicit heap-dynamic

Copyright © 2012 Addison-Wesley. All rights reserved.

1. Static Variables

- Bound to memory cells before execution
  - remains bound to the same memory cell throughout execution
- **Applications:**
  - globally accessible variables, to have them bound to same storage during execution
  - to make some variables of subprograms to keep values same between separate execution of the subprogram (history sensitive subprograms)

Copyright © 2012 Addison-Wesley. All rights reserved.

1. Static Variables

- **Advantage:** efficiency
  - direct addressing (no run-time overhead for allocation and deallocation).
- **Disadvantages:**
  - reduced flexibility
    - if a programming language has only static variables, it cannot support recursion
  - storage cannot be shared among variables
- **Examples:**
  - Static variables in C, C++ and Java

Copyright © 2012 Addison-Wesley. All rights reserved.

2. Stack-Dynamic Variables

- Storage bindings are created for variables when their declaration statements are *elaborated*.
  - A declaration is elaborated, when execution reaches the code where the declaration is done
  - **Example**: variable declarations in a Java method are elaborated when the method is called and deallocated when the method completes its execution

Copyright © 2012 Addison-Wesley. All rights reserved.

2. Stack-Dynamic Variables

- **Advantages:**
  - allows recursion;
  - same memory cells can be used for different variables
- **Disadvantages:**
  - Overhead of allocation and deallocation
  - Subprograms cannot be history sensitive
  - Slower accesses (indirect addressing)

Copyright © 2012 Addison-Wesley. All rights reserved.

2. Stack-Dynamic Variables

- In Java, C and C++,
  - local variables in methods are, by default, stack-dynamic,
  - but can be made static through static qualifier.

```
foo () {
  static int x;
  ...
}
```

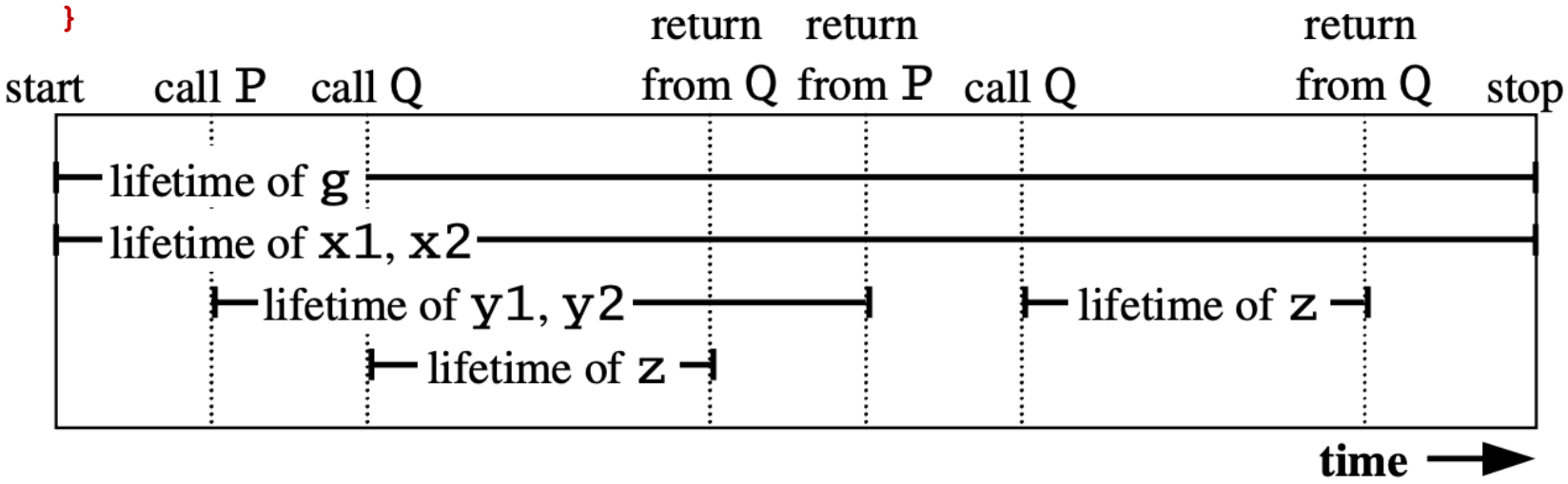
Copyright © 2012 Addison-Wesley. All rights reserved.

Lifetime

```
int g;
void main () {
    int x1;
    float x2;
    ...
    P();
    ...
    Q();
    ...
}
```

```
void P () {
    float y1;
    int y2;
    ...
    Q();
    ...
}
```

```
void Q () {
    int z;
    ...
}
```

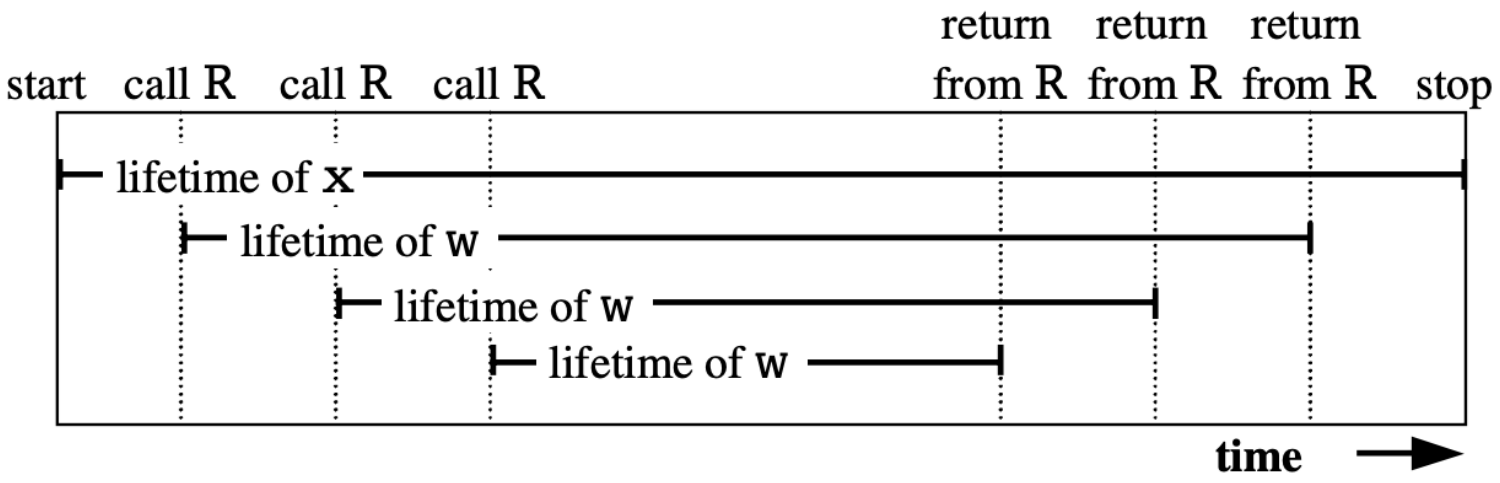


Copyright © 2012 Addison-Wesley. All rights reserved.

Lifetime (recursive function)

```
void main () {
    int x;
    ...
    R();
    ...
}
```

```
void R () {
    int w;
    ...
    R();
    ...
}
```



Copyright © 2012 Addison-Wesley. All rights reserved.

3. Explicit Heap-Dynamic Variables

- Allocated (and deallocated) by explicit instructions, written by the programmer
- Referenced only through pointers or references,
  - all objects in Java
  - dynamic objects in C++ (via `new` and `delete`),

```
int *intnode;
intnode = new int;
…
delete intnode;
```

  - C# has both stack-dynamic and explicit heap-dynamic objects

Copyright © 2012 Addison-Wesley. All rights reserved.

3. Explicit Heap-Dynamic Variables

- static type binding: type binding at compile time
- dynamic storage binding: memory cell is allocated when created explicitly
- Advantages:
  - Required for dynamic structures that grow/shrink during execution (e.g., linked lists, trees)
- Disadvantages:
  - Difficult to use correctly
  - Costly to refer, allocate, deallocate

Copyright © 2012 Addison-Wesley. All rights reserved.

4. Implicit Heap-Dynamic Variables

- Allocation and deallocation caused only by assignment statements
  - Storage and type bindings are done when they are assigned values
- all strings and arrays in Perl, JavaScript, and PHP
- Advantage:
  - highest degree of flexibility (generic code)
- Disadvantages:
  - Runtime overhead for allocation and deallocation
  - Loss of error detection

Copyright © 2012 Addison-Wesley. All rights reserved.

Binding Process

- Language processor keeps track of the current environment in a data structure called Symbol Table or Identifier Table
- Symbol table maps identifier names to their type and binding
- Each new block introduces its declarations/bindings to the symbol table and on exit, they are cleared
- For static binding, symbol table is a compile time data structure and maintained during different stages of compilation
- For dynamic binding, symbol table is maintained at run time

Copyright © 2012 Addison-Wesley. All rights reserved.

The `LET` Construct

- Most functional languages include some form of `let` construct that is related to the blocks of imperative languages

- A let construct has two parts
  - The first part binds names to values
  - The second part uses the names defined in the first part

Copyright © 2012 Addison-Wesley. All rights reserved.

The LET Construct (continued)

- In Haskell:

```
let
  val name1 = expression1
  …
  val namen = expressionn
in
 expression
end;
```

- Ex:

```
let exp1 = 2 * (let x1 = 20 in x1 + 400) + 8
```

Copyright © 2012 Addison-Wesley. All rights reserved.

The LET Construct (continued)

- In Scheme:

```
(let (
  (name1 expression1)
  …
  (namen expressionn))
 expression
)
```

```
Ex: (let (
        ( top (+ a b) )
        ( bottom (- c d) ) )
    ( / top bottom )
    )
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Declaration Order

- C99, C++, Java, and C# allow variable declarations to appear anywhere a statement can appear
  - In C99, C++, and Java, the scope of all local variables is from the declaration to the end of the block
  - In C#, the scope of any variable declared in a block is the whole block, regardless of the position of the declaration in the block
    - However, a variable still must be declared before it can be used

Copyright © 2012 Addison-Wesley. All rights reserved.

Declaration Order (continued)

- In JavaScript, local variables can be declared anywhere in a function
  - The scope of such variable is always the entire function
  - If used before its declaration in the function, the value will be `undefined`
- In C++, Java, and C#, variables can be declared in `for` statements
  - The scope of such variables is restricted to the `for` construct

Copyright © 2012 Addison-Wesley. All rights reserved.

Global Scope

- C, C++, PHP, and Python support a program structure that consists of a sequence of function definitions in a file
  - These languages allow variable declarations to appear outside function definitions
- C and C++have both declarations (just attributes) and definitions (attributes and storage)
  - A declaration outside a function definition specifies that it is defined in another file

Copyright © 2012 Addison-Wesley. All rights reserved.

Global Scope – PHP

- PHP
  - Programs are embedded in HTML markup documents, in any number of fragments, some statements and some function definitions
  - The scope of a variable (implicitly) declared in a function is local to the function
  - The scope of a variable implicitly declared outside functions is from the declaration to the end of the program
    - Global variables can be accessed in a function through the `$GLOBALS` array or by declaring it `global`

Copyright © 2012 Addison-Wesley. All rights reserved.

Global Scope – PHP (continued)

- PHP

```
$day = "Monday";
$month = "January";
function calendar() {
    $day = "Tuesday";
    global $month;
    print "local day is $day \n";
    $gday = $GLOBALS['day'];
    print "global day is $gday \n
    print "global month is $month \n";
}
calendar();
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Global Scope – PHP (continued)

- PHP

```
$day = "Monday";
$month = "January";
function calendar() {
    $day = "Tuesday";
    global $month;
    print "local day is $day \n";
    $gday = $GLOBALS['day'];
    print "global day is $gday \n";
    print "global month is $month \n";
}
calendar();
```

OUTPUT

local day is Tuesday

global day is Monday

global month is January

Copyright © 2012 Addison-Wesley. All rights reserved.

Global Scope – Python

A global variable can be referenced in functions, but can be assigned in a function only if it has been declared to be global in the function

1)
```
day = "Monday"
def tester():
    print("The global day is:", day)
tester()
```
OUTPUT: `The global day is: Monday`

2)
```
day = "Monday"
def tester():
    print("The global day is:", day)
    day = "Tuesday"
    print("New val of day is:", day)
tester()
```
OUTPUT: `UnboundLocalError`

3)
```
day = "Monday"
def tester():
    global day
    print("The global day is:", day)
    day = "Tuesday"
    print("New val of day is:", day)
tester()
```
OUTPUT:
```
The global day is: Monday
New val of day is: Tuesday
```

Scope and Lifetime

- Scope and lifetime are sometimes closely related, but are different concepts

```
void printheader() {
   . . .
} /* end of printheader */
void compute() {
   int sum;
   . . .
   printheader();
   . . .
} /* end of compute */
```

Copyright © 2012 Addison-Wesley. All rights reserved.